between probability distributions; it tells you how different two probability distributions are.

Often we are trying to minimize the distance between a model-generated probability distribution and some empirical distribution from real data, so we want to minimize the KL divergence. As you just saw, minimizing the KL divergence is equivalent to maximizing the joint log-likelihood ratio of the generated data compared to the empirical data. One important thing to note is that the KL divergence is not symmetric, i.e., $D_{KL}(Q \| P) \neq D_{KL}(P \| Q)$, and this should be clear from its mathematical definition. The KL divergence contains a ratio, and no ratio can equal its inverse unless both are 1, i.e., $\frac{a}{b} \neq \frac{b}{a}$ unless $a = b$.

Although the KL divergence makes a perfect objective function, we can simplify it just a bit for our purposes. Recall that $\log(a/b) = \log(a) - \log(b)$ in general. So we can rewrite the KL divergence as

$$D_{KL}(Q \| P) = -\Sigma_i Q(x) \times \log(P(x_i)) - \log(Q(x_i))$$

Note that in machine learning, we only want to optimize the model (update the parameters of the model to reduce the error); we cannot change the empirical distribution $Q(x)$. Therefore, we really only care about the weighted log probability on the left side:

$$H(Q,P) = -\Sigma_i Q(x) \times \log(P(x_i))$$

This simplified version is called the cross-entropy loss and is denoted $H(Q,P)$. This is the actual loss function that we will use in this chapter to get the error between our predicted action-value distribution and a target (empirical) distribution.

In listing 7.8 we implement the cross-entropy loss as a function that takes a batch of action-value distributions and computes the loss between that and a target distribution.

Listing 7.8. The cross-entropy loss function